

Machine Learning Technologies

Week 1 Introduction

Donny Hurley

What is Machine Learning (ML)?

- Algorithms that automatically improve performance through experience
- Often this means define a model by hand, and use data to fit its parameters

There are problems that are difficult for humans but easy for computers

- E.g. calculating large arithmetic problems

And there are problems easy for humans but difficult for computers

- E.g. recognising a picture of a person from the side

Machine Learning often tries to leverage the things computers are good at (large arithmetic and repetitive problems) to solve things that we are usually able to do more easily.

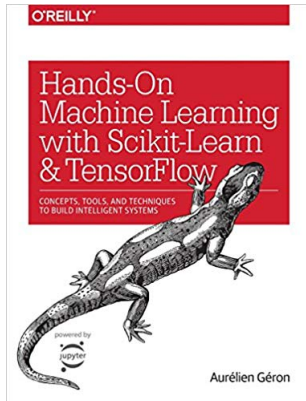
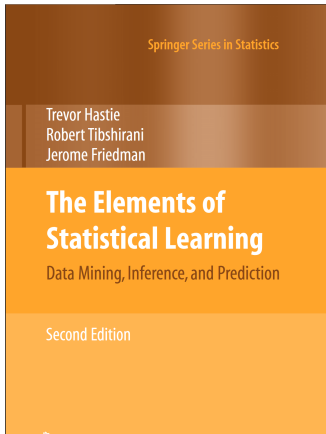
Why ML?

- The real world is complex – difficult to hand-craft solutions.
- Think about how many "if" statements would be needed!
- ML is the preferred framework for applications in many fields:
 - Computer Vision
 - Natural Language Processing
 - Speech Recognition
 - Robotics
 - ...
- Humans can typically parse sentences but computers do not get the context.
Humans are not consistent with grammar etc but computers expect consistency.

There are absolutely loads of books on machine learning



Joel Grus



General ML stuff:

- Supervised and Unsupervised Algorithms
- Classification
- Regression
- Generalisation
 - Underfit/Overfit
 - Bias/Variance
 - Training/Test sets
 - Error/Loss Functions
- Cross-Validation
- Gradient Descent
- Regularisation

Models/Algorithms:

- Naive Bayes
- K-Nearest Neighbour
- Support Vector Machines
- Neural Networks
- Principle Component Analysis

Applications:

- Natural Language Processing (NLP) (Basic not LLMs!)
- Image Recognition
- Some other things

Three components for assessment:

- Either an in-class lab exam or project about shallow AI around week 7 (30%)
- A project about Convolutional Neural Networks due at end of semester (30%)
- An Exam (40%) during exam time

Basic idea of all machine learning

Machine Learning is using an **algorithm** that can learn from **Data**.

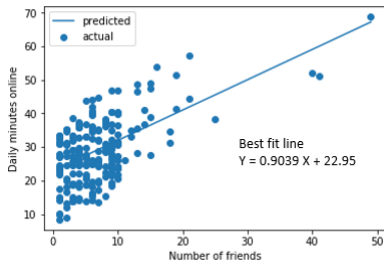
We want to create a **Model** that takes input and gives output. It is basically

$$y = f(x)$$

We figure out the right-hand side of the equation by gathering a lot of data and making the *parameters* best fit our base model.

Basic Example

Take a base model of $y = w_0 + w_1x$, we need to figure out the "best" w_0 and w_1 parameters e.g.



Of course our base model can be much more complicated, have many more parameters that need to be learnt, but the basic idea is the same.

After we have made the model, we can then make *predictions* in the future. Above, how many daily minutes do we predict a person with 30 friends on the platform?

Why does it take so long?

In the above example, one parameter, a basic line as the model, learning all the parameters is very very quick. But real world is not that quick.

Some ML models can takes hours, days, weeks or even months to be *trained*. ChatGPT (some version of it anyway!) is actually 8 different models, with 220 billion parameters in each. This takes a very long time to figure out.

Why does it take so long?

The *training* is repeatedly doing the same procedure over and over and over and over again, until we get the parameters we think are best.



Most of the data we want, needs to be *labelled*. This means, any data we put into the system is a pair of information

$$y, \mathbf{x}$$

where y is the right answer for that and $\mathbf{x}$ is the collection of all features

So let's say we have a collection of images that are cats or dogs and we want to build a model that can separate them.

For a particular image there is a correct y , i.e. y is cat or dog and then $\mathbf{x}$ is all the features of the image i.e. each x will be the colour of a pixel.

Dataset example

We (the human intelligence) must do a lot of work here.

We need lots of training data and labels associated with them.

Example: MNIST Data set.

To learn to recognise handwritten letters the algorithm must be trained on 60,000 samples, each of which must be labelled correctly - by hand.



Samples of data set, from wikimedia.

Hand-written Digit Recognition

Now let's look at some potential applications and how we could try "phrasing" the problem.



Difficult to hand-craft rules about digits.

Hand-written Digit Recognition



$$x_i = \text{[image]}, y_i = (0, 0, 0, 0, 1, 0, 0, 0, 0, 0)$$

So the image can be represented by the value of each pixel. Each pixel colour is called a *feature*, they are independent variables..

- Represent input image as a vector $x_i = \mathbb{R}^{784}$
- Suppose we have a target vector t_i
 - This is supervised learning
 - Discrete, finite label set: a classification problem.
- Given a training set $\{(x_1, y_1), \dots, (x_N, y_N)\}$, the learning problem is to construct a "good" function $y(x)$ from these.
 - $y : \mathbb{R}^{784} \rightarrow \mathbb{R}^{10}$

Some algorithms will not exactly return $(0, 0, 0, 1, \dots)$ etc. but may return probabilities like $(0.01, 0.03, 0.04, \dots, 0.01, 0.88 \dots)$ so the image is *most likely* a 4.

Face Detection



Schneiderman and Kanade, IJCV 2002

- Classification problem.
- $t_i \in \{0, 1, 2\}$, non-face, frontal face, profile face.

Of course this can be expanded into the face of a particular person so the possible t_i set could be large. t_i is the set of ALL possible results.

We **map/transform** the image to a particular t_i

Stock Price Prediction



- Problems in which t_i is continuous are called **regression**
- E.g. t_i is stock price, x_i contains company profit, debt, cash flow, gross sales, number of spam emails sent, . . .

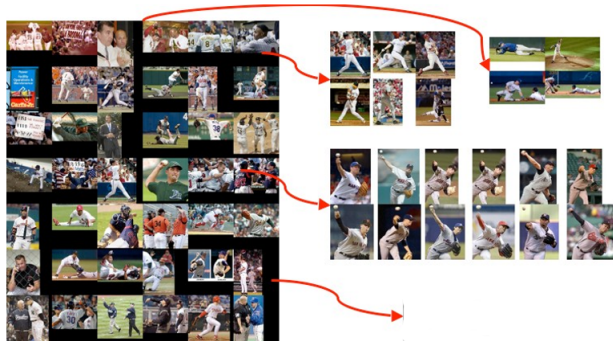
Techniques where we have training examples where we **know** the correct result (the y values). Different types of supervised learning are :

- Classification
- Regression

Some algorithms :

- Linear/Logistic Regression
- Naive Bayes
- k-Nearest Neighbours
- Support Vector Machines
- Neural Networks
- Decision Trees
- Random Forests

Clustering Images



Wang et al., CVPR 2006

- Only x_i is defined : **unsupervised learning**
- E.g. x_i describes image, find groups of similar images

Techniques where there is no "right" answer known but the algorithm tries to find some **structure/patterns** in the data.

- Principal Component Analysis (Dimensionality Reduction)
- k-mean Clustering
- PageRank (Google)

Unsupervised learning techniques will group things together but we do not necessarily know what the groups are. The images example, after the grouping was done we were able to say "This group is pitching, this group is sliding " etc. But we had no way of knowing this before it was run and the computer does not have labels for the data after the algorithm.

There are other types of Machine Learning, e.g. Reinforcement Learning, but we won't be talking about them at all in this course. If you've tried using ML-Agents in Unity, these are an example of Reinforcement Learning.

We won't be getting as far as Recurrent Neural Networks (often used for time-series data, sometimes text data) or things like LSTM style models.

Additionally we won't be doing anything with Transformer models (these are the breakthrough that made GPTs work and it has been discovered to be very good for other things too - they are more complex for training and using)

What is the one big problem with supervised learning techniques?

All the training data has to be labelled, often manually, before the model can be built.

Unsupervised learning: It is often easier to obtain unlabeled data - from a lab instrument or a computer - than labeled data, which can require human intervention.

E.g. Netflix do not have defined groups of people but they group people into clusters and make recommendations based on the cluster they are in.

Types of Machine Learning

	Supervised Learning	Unsupervised Learning
Discrete	classification or categorization	clustering
Continuous	regression	dimensionality reduction

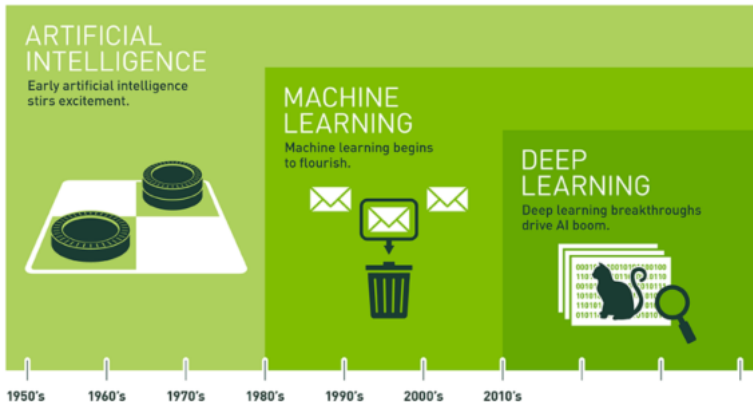
- Often referred to as narrow AI.
- We are only concerned with a *specific task*.
- So the goal of machine learning is not to develop general intelligence or a universal learning algorithm.
- Machine learning seeks an algorithm that learns a particular task well
- And should result in a system that *probably* carries out the task correctly on *most* occasions.

Almost every AI thing talk about at the moment are really examples of Machine Learning (ML) which is a subset of AI.

ML is *Narrow AI*, it is designed for a specific task. Ask ChatGPT if it is an example of narrow AI and it says

Yes, as an AI language model developed by OpenAI, I am an example of narrow AI. I am designed for a specific task, which is generating human-like text based on the input I receive from users. My capabilities are focused on natural language understanding and generation, and I am not capable of general intelligence or tasks outside of my specific domain.

AI - Machine Learning - Deep Learning

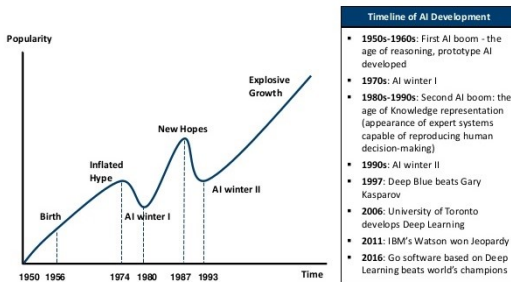


OpenDataScience
Medium
NewGenApps

What is Machine Learning (ML)?

- We tell the system *how* to learn not what to do.
- Many of the learning algorithms have been around for decades going right back to the 1960s.
- Neural Networks had beginnings in the 1940s, Deep Learning is building upon "basic" neural networks.
- Rosenblatt (1958) created the perceptron, the building blocks of Neural Networks.
- The problem was, they didn't work very well. Research stagnated as basic perceptrons were incapable of processing the exclusive-or circuit and that computers lacked sufficient power to process useful neural networks.
- This has been one of the problems with AI in general.
- Something new is developed that seems full of promise but then turns out to not work very well in practice.
- So AI goes back into hibernation...

AI HAS A LONG HISTORY OF BEING “THE NEXT BIG THING” ...



Source <https://www.actuaries.digital/2018/09/05/history-of-ai-winters/>

What has changed?

Two major developments have brought all the machine learning algorithms back into fashion, especially Neural Networks/Deep Learning.

What seemed like insurmountable problems in the past now maybe have solutions

- *Big Data*

- The amount of data available now is orders of magnitude larger than the past.
- A "rule" of ML, the more data the better.
- Neural Networks/Deep Learning in particular needs a lot of data to work properly.
- In the past the issue was trying to get more data, now it is often, how do we process all this information.

- *Compute Power*

- In particular GPUs
- People realised the efficiency and power of graphics cards (thanks gamers!) for doing parallel calculations of maths that ML relies on.
- These graphics cards can do matrix (Linear Algebra) operations much more efficiently than a CPU. (Tensor maths, Tensor cores, Tensor Flow)

I talked about more historical things on the earlier slides, what have been the *big* breakthroughs that led us to where we are now

- 2012: AlexNet paper and algorithm achieves breakthrough results in image recognition in the ImageNet benchmark. This popularizes deep neural networks.
- Rest of 2010s: Further refinement of classification algorithms and improvements for the ImageNet benchmarks and others
- 2017: A team at Google Brain invent the transformer architecture. This is widely used for Text processing so led to LLMs, but also used by Nvidia for newer DLSS models.
- 2020s: GenAI took off this decade.

What do we need, to do Machine Learning?

- The Task - define or limit the task
- The Experience - This is the data, more data = more experience
- The Performance measure - We need a good way to measure this.
- The Learning Algorithm - The recipe by which we will improve our performance.
- The Intelligence - the Network - the brain.

What's in the following image?

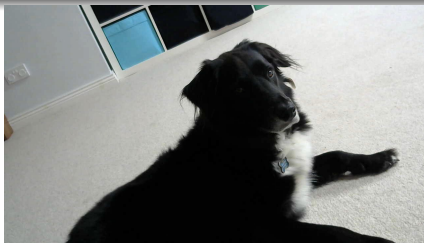


How would you instruct a computer to recognise what's in the image?

The computer doesn't need to be able to define what a dog is, in order to classify it.

We give it lots of pictures of dogs, telling the computer: these are pictures of dogs. It is then up to the computer to “decide” how to classify based on the algorithm. What parts does it pick out as important.

Alternatively



Can a computer tell the difference between these two dogs?

The computer doesn't need to be able to define who each dog is, in order to classify it. We give it lots of pictures of the dogs, telling the computer: these are pictures of two different dogs. It is then up to the computer to “decide” how to classify based on the algorithm.

Again, what parts does it pick out as important.

An ML Model does not need to explain itself, so it may be taking completely different information than you expect e.g. <https://venturebeat.com/business/when-ai-flags-the-ruler-not-the-tumor-and-other-arguments-for-abolishing->

<https://github.com/daviddao/awful-ai> keeps a collection of Awful AI.

Machine Learning Technologies

Regression

Donny Hurley

Independent/Dependent Variables

Short note on *independent* and *dependent* variables

Example

The Value of a House Over Time

Here we have two variables

- ① *Value of House (V)*
- ② *Time (t)*

Since they both change, they are Variables

- **Value** is a **Dependent** variable, since the value of a house **depends on** the **time** you bought it
- **Time** is **independent**.
- Plotting we typically leave the **horizontal axis** to represent the **independent** variable and the **vertical** axis to represent the **dependent** variable

Linear Relationships

If this course, we will mostly look at linear relationships to make our predictions. This should be revision on the next couple of slides!

Definition

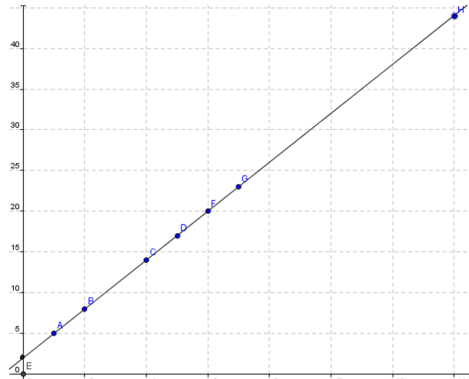
A Linear relationship: a change in the value of x always produces the same proportionate change in the value of y .

Consider the following table:

x	1	2	3	4	5	6				
y	5	8		14	17	20				44

- a What is the value of y when $x = 3$?
- b What value of x will give $y = 44$?

- Once we are convinced that some relationship does exist, we can establish the precise nature of that relationship and use it to predict values of one variable that would correspond to any given values of the other.
- The exactness of the relationship can be seen in the diagram above. The plotted points lie along an imaginary straight line. If we were to **draw a straight line through them, we could use it for making exact predictions without even using the formula.**



Can you come up with a formula for the previous graph???

$$y = mx + c$$

$$y = 3x + 2$$

for this particular line. We can now use this formula to "predict" values.

This very very early taught equation of a line is the basis for regression and lots of machine learning analysis

This is where we try to zone in on a continuous value(s) rather than trying to predict a class or category.

So, for example if we are trying to predict the distance to a pedestrian, from a vehicle, this would be a regression problem.

Example: How to Cook a Turkey

There is a rule of thumb when cooking a turkey.

Put it in a pre-heated oven for 20 minutes per pound plus 20 minutes.

Where did this idea come from? Is it correct? What could happen if it was wrong?

Example: How to Cook a Turkey

This is a linear model which is likely based on linear regression analysis of different weights of turkeys. The mathematical model is

$$t = mw + c$$

where t is the cooking time, w is the weight in pounds, m is the slope of the line and c is the y-intercept.

Pief Panofsky disagreed and used the following model.

$$t = w^{(2/3)} / 1.5$$

Let's analyse this.

Example: How to Cook a Turkey

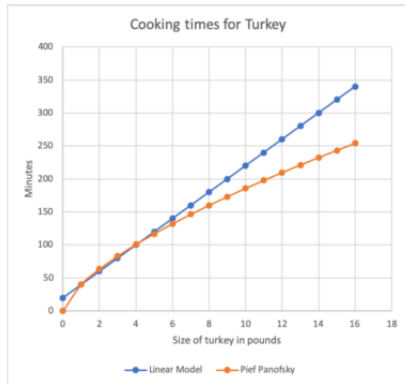
First notice that for the linear model you have to cook an imaginary turkey for 20 minutes.

Between 1 pound and 6 pounds, the models are very similar. There is 90 minutes difference between them at 16 pounds.

Which is Safer?

Which tastes better?

Use a meat thermometer.



Fitting a linear model

With the above example we see the result of a linear model but not how it was arrived at.

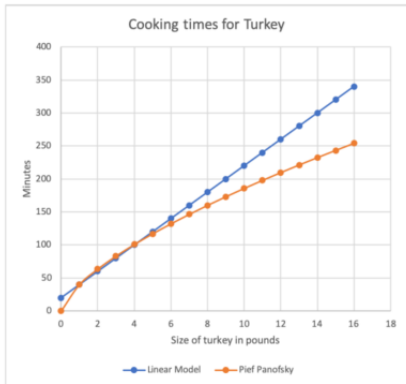
To do that we have to collect some data, and fit the *best* line to it.

In the above case this would mean cooking many turkeys of various weights and cooking them all using some calibrated method, such as a meat thermometer (or an expert cook with experience).

We collect the "right" answer and use this to build the model i.e. figure out the parameters.

Let's look at some data.

Example: How to Cook a Turkey



Given the data to the left, if you had a nine pound turkey, how long should you cook it for?

Is this a task for supervised learning or unsupervised learning?

If we assume a linear model, what are the parameters that we need to learn?

$$t = mw + c$$

Example: How to Cook a Turkey

Weight in Pounds	1.1	1.3	1.7	2	2.1	2.3	...
Time taken to cook	39.4	48.8	55.8	63.0	69.5	67.7	...

So w is our input variable/feature.

t is our output and can be considered our regression target that we will use to train the model.

We need to learn m and c .

We will also use the notation of M for number of training samples $M=83$ in this case.

(w, t) one training example.

$(w^{(i)}, t^{(i)})$ the i -th training example.

You will notice in the previous slides that we used variable names that either match the problem or that are used regularly in Mathematics.

However, we will not always be dealing with linear models so we are going to use notation that can be scaled up to larger models.

There isn't a set notation, so where possible I will use whatever is used in scikit-learn

Therefore we will use w to represent parameters/weights that we are going to train.

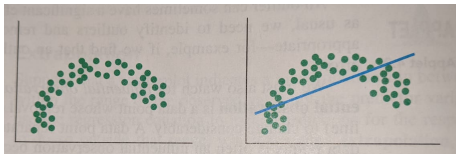
We will most often use y as our output and x as input.

So our above equation will now be

$$y = w_0 + w_1x$$

Warning on Use of Linear Regression

Linear Regression is based on the assumption that the data points are scattered about a line. However, often data points are scattered on a curve, example



You can still compute the coefficients (β_0 and β_1) which will give the line above. However, we can see it is an inappropriate fit as the line suggests the y-values keep increasing while the curve shows that they will actually decrease after a point

Criterion for Finding a Regression Line

Before finding a regression line for a set of data points, draw a scatterplot. If the data points do not appear to be scattered about a line, do not determine a regression line.

This was an example of a *Simple Linear Regression* model. There is only one independent feature, but of course the real world doesn't work like that. There can be polynomial regression

$$y = w_0 + w_1x + w_2x^2 + \dots w_nx^n$$

Multiple Linear Regression, where instead of x being a single feature it is a vector of features $\mathbf{x} = (x_1, x_2, x_3, x_4, \dots)^T$

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots$$

We will look at those in some detail later, but keep in mind the *weights* (w 's) are still linear in both instances. We can also write the weights as a vector

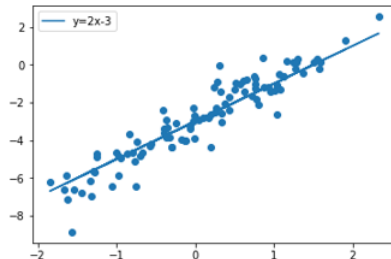
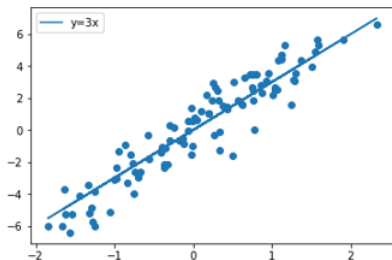
$$\mathbf{w} = (w_0, w_1, w_2, w_3, w_4, \dots)^T$$

What you know about Simple Linear Regression can be expanded to the other cases.

Errors

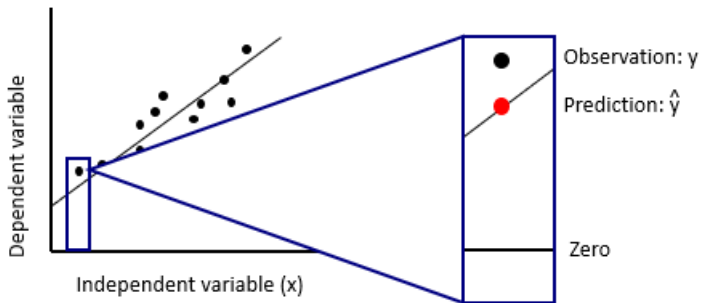
The first example I put up, had all the data points (training data) fall exactly on the line. This is not how it works really. The prediction is not going to be exactly the same as what we fed into our model to train. Think of the example I put up about the Friends vs Daily Minutes Online

Example



So there is some amount of *error* on the model vs the real data.

Simple Linear Regression



- The function will make a prediction for each observed data point.
- The observation is denoted by y and the prediction is denoted by $\hat{y}$.

Here's an idea, let's find the line that has the smallest amount of error and that will be the model we want!

Training the Model

Training the model means, find the best line that minimises the error for our training sample. This depends on the *Loss* function.

Measuring the difference between what the model predicts the value should be and what the training sample actually was.

There are multiple loss functions to choose and choosing the wrong one can be like choosing the wrong type of model.

Typical example is the mean squared error

$$L = \frac{1}{2m} \sum_{i=0}^m (\hat{y}_i - y_i)^2$$

where m is the number of samples, $\hat{y}_i$ is the prediction for the i -th sample and y_i is the correct answer for that sample.

Note : the $\frac{1}{2m}$ is just chosen for convenience as the Maths will look "nicer". Even still minimising L without the fraction will give us the same values for w .

Training the model

So what we want are the particular parameters (say w_0 and w_1 in the simplest case we've seen) that give us the smallest value for L .

This is actually quite easy for the linear regression examples we've had so far, there's a known formula, or we can use some algebra.

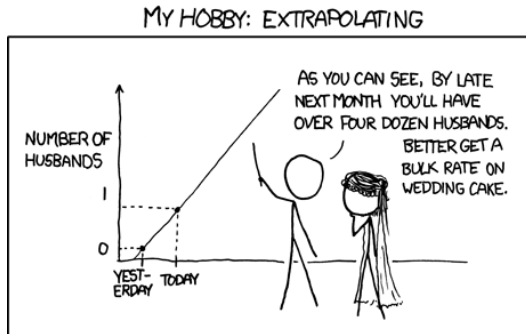
But we want the idea to work for much more complicated models too, and with more parameters.

The idea does work in further ML algorithms, get the parameters $w_0, w_1, w_2, \dots$ that give us the smallest L , just the technique for finding the parameters might take a bit longer in more complicated models. This is even true of deep learning (Neural Networks).

Usually we use a method called gradient descent to solve this (later!), that does multiple iterations to find the best parameters. When we use the `.fit()` methods in scikit-learn and tensorflow this is what they do.

Extrapolation

- Extrapolation is a type of estimation, beyond the original observation range, the value of a variable on the basis of its relationship with another variable.
- Interpolation, which produces estimates between known observations
- Extrapolation is subject to greater uncertainty and a higher risk of producing meaningless results.



Building a model

We have a set of data, that is a pair of information. We know the correct answer for the set.

- X is the set of all inputs (a matrix).
- y is the set of all corresponding response variables.
- A particular y_i is the result for a particular x_i .

```
from sklearn import linear_model

lr = linear_model.LinearRegression()
lr.fit(X, y)
pred = lr.predict(X)
```